



รายงาน

เรื่อง การสร้างสมการพหุนามด้วยเมทริกซ์จากข้อมูลการปล่อยโดรน

โดย

นายมีโชค ประพันธ์เจริญสิน
รหัสนักศึกษา 68050449

เสนอ

รศ. อีรวัฒน์ ประกอบผล

รายงานนี้เป็นส่วนหนึ่งของรายวิชา ฟิสิกส์เชิงเส้นสำหรับวิทยาการ
คอมพิวเตอร์(05506250)

สารบัญ

บทนำและที่มาของข้อมูล (Story)	3
ข้อมูลที่บันทึกได้	3
สร้างสมการพหุนาม	4
โปรแกรมแสดงความสัมพันธ์	5
อธิบายการทำงานของโปรแกรม/Flowchart	7
วิธีการใช้งานโปรแกรม	8
ผลลัพธ์จากการรันโปรแกรม	9
ข้อสรุป	9

บทนำและที่มาของข้อมูล (Story)

- ในการพัฒนาระบบควบคุมการบินโดรนเพื่อการเกษตร บริษัท “ AgriSky Solution” ได้ทำการทดสอบลำดับการปล่อยโดรนขึ้นบิน ของโดรนที่ใช้สำหรับตรวจแปลงเพาะปลูก โดยระหว่างการทดสอบ วิศวกรได้ติดตั้งเซนเซอร์วัดความดันเพื่อบันทึกความสูงของโดรนทุกๆ 0.5 - 1 วินาที นับตั้งแต่ที่มอเตอร์เริ่มทำงานจนถึงวินาทีที่ 5 หลังการปล่อย
- จุดประสงค์ของการเก็บข้อมูลนี้คือ ต้องการสร้างแบบจำลองทางคณิตศาสตร์ ที่อธิบายความสัมพันธ์ระหว่างเวลา (X) และความสูง (Y) ของโดรนในช่วงการไต่ระดับเพื่อนำไปใช้พยากรณ์ความสูง ณ เวลาใดๆ และปรับปรุงอัลกอริทึมควบคุมการบินให้แม่นยำขึ้น
- จากลักษณะของการไต่ระดับในแนวดิ่งข้อมูลที่ได้จึง มีแนวโน้มเป็นความสัมพันธ์แบบไม่เชิงเส้น

ข้อมูลที่บันทึกได้

- ตารางข้อมูลเวลา (X) และความสูง (Y) ของโดรนที่บันทึกได้

ลำดับ	เวลา (วินาที)	ความสูง (เมตร)
1	0.0	0.00
2	0.5	1.05
3	1.0	1.95
4	1.5	3.30
5	2.0	4.65
6	2.5	6.40
7	3.0	8.15
8	3.5	10.30
9	4.0	12.40
10	4.5	15.05
11	5.0	17.55

สร้างสมการพหุนาม (Least Squares Polynomial Fitting)

จำนวนจุดข้อมูล 11

ผลรวม x 27.5

ผลรวม x² 96.25

ผลรวม x³ 378.125

ผลรวม x⁴ 1583.3125

ผลรวม y 80.8

ผลรวม xy 298.3

ผลรวม x²y 1209.675

- เขียนเป็นระบบสมการปกติ

$$11A + 27.5B + 96.25C = 80.8 \quad (1)$$

$$27.5A + 96.25B + 378.125C = 298.3 \quad (2)$$

$$96.25A + 378.125B + 1583.3125C = 1209.675 \quad (3)$$

- กำจัดตัวแปร

กำจัด A ออกจากสมการ (2) โดย (2) - (2.5*(1)) :

$$27.5B + 137.5C = 96.3 \quad \text{-----(4)}$$

กำจัด A ออกจากสมการ (3) : โดย (3) - (8.75*(1))

$$137.5B + 741.125C = 502.675 \quad \text{-----(5)}$$

กำจัด B ออกจากสมการ (4) : โดย (4) - (5*(3))

$$53.625C = 21.175 \quad \text{-----(6)}$$

- แทนย้อนกลับ

หา C จาก (6) :

$$C = \frac{21.175}{53.625} = 0.395$$

หา B จาก (3) แทน C = 0.395 :

$$B = \frac{96.3 - 54.295}{27.5} = 1.527$$

หา A จาก (1) แทน B = 1.527 , C = 0.395 :

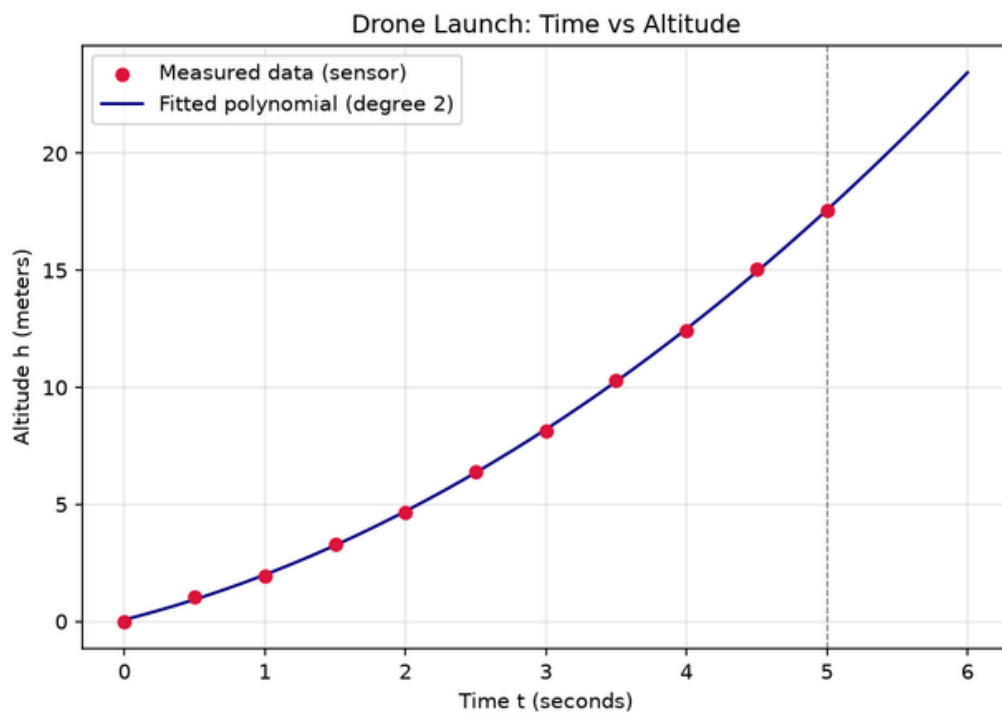
$$A = \frac{80.8 - 42.005 - 38.006}{11} = 0.072$$

- จะได้สมการ :

$$Y = 0.072 + 1.527x + 0.395x^2$$

X คือ เวลา t t หลังปล่อยโดรน หน่วยเป็นวินาที

Y คือ ความสูงของโดรน หน่วยเป็นเมตร ณ เวลานั้น



ภาพนี้แสดงถึงความสัมพันธ์ของเวลากับความสูงของโดรน

โปรแกรมแสดงความสัมพันธ์

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def build_vandermonde(x, degree):
5     n = len(x)
6     A = np.zeros((n, degree + 1))
7     for i in range(n):
8         for j in range(degree + 1):
9             A[i][j] = x[i] ** j
10    return A
11
12
13 def normal_equation_matrices(A, y):
14     At = A.T
15     AtA = At @ A
16     Aty = At @ y
17     return AtA, Aty
18
19
20 def gaussian_elimination(M, b):
21     k = len(b)
22     Aug = np.hstack([M.astype(float), b.reshape(-1, 1).astype(float)])
23     for col in range(k):
24         pivot_row = np.argmax(np.abs(Aug[col:, col])) + col
25         if Aug[pivot_row, col] == 0:
26             raise ValueError("เมทริกซ์เอกฐาน (singular matrix) ไม่สามารถแก้ได้")
27         Aug[[col, pivot_row]] = Aug[[pivot_row, col]]
28         for row in range(col + 1, k):
29             factor = Aug[row, col] / Aug[col, col]
30             Aug[row, col:] -= factor * Aug[col, col:]
31     c = np.zeros(k)
32     for row in range(k - 1, -1, -1):
33         c[row] = (Aug[row, -1] - Aug[row, row + 1:k] @ c[row + 1:k]) / Aug[row, row]
34     return c
35

```

```

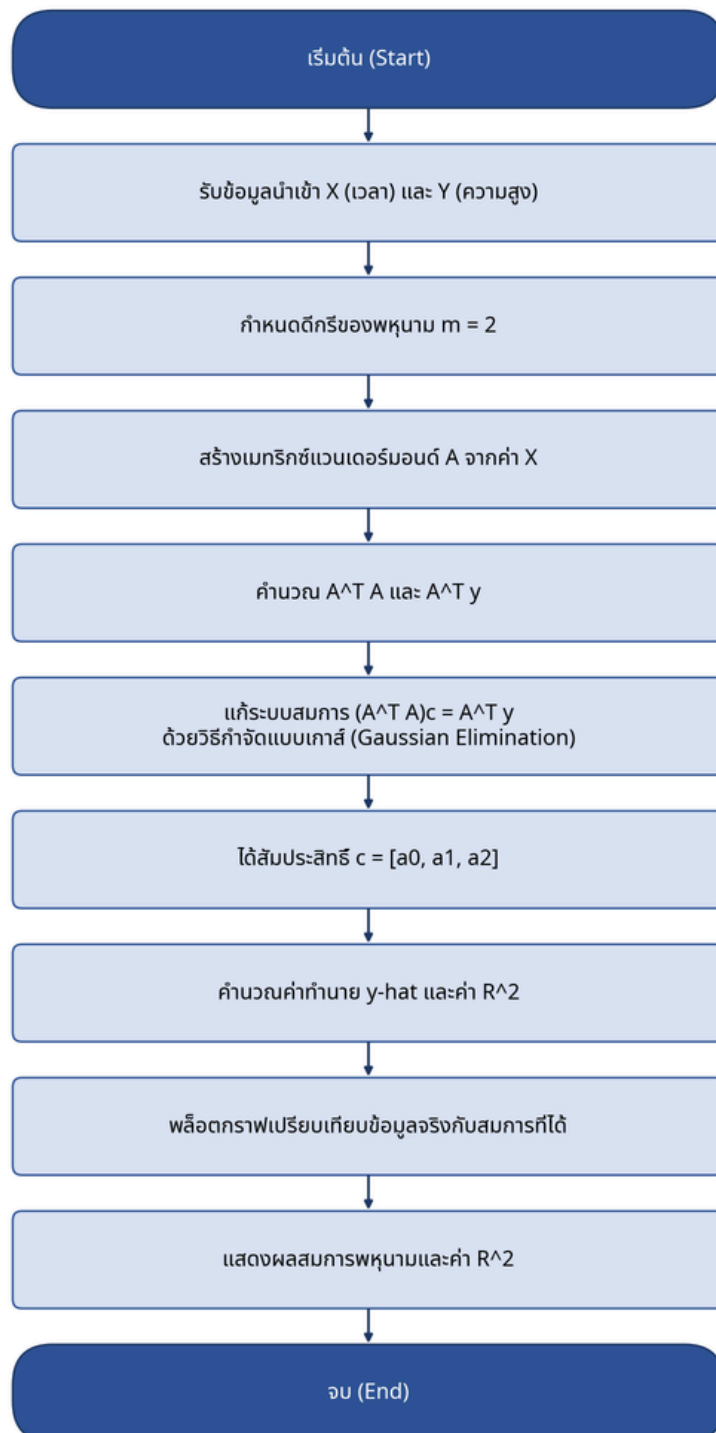
36
37 def r_squared(y, y_pred):
38     ss_res = np.sum((y - y_pred) ** 2)
39     ss_tot = np.sum((y - np.mean(y)) ** 2)
40     return 1 - ss_res / ss_tot
41
42
43 def format_polynomial(coeffs, decimals=3):
44     terms = []
45     for j, a in enumerate(coeffs):
46         if j == 0:
47             terms.append(f"{a:.{decimals}f}")
48         elif j == 1:
49             terms.append(f"{a:.{decimals}f}x")
50         else:
51             terms.append(f"{a:.{decimals}f}x^{j}")
52     return "y = " + " ".join(terms)
53
54
55 def predict(coeffs, x_new):
56     return sum(a * (x_new ** j) for j, a in enumerate(coeffs))
57
58 if __name__ == "__main__":
59     # X = เวลาหลังปล่อยโดรน (วินาที)
60     # Y = ความสูงที่วัดได้จากเซนเซอร์วัดความดัน (เมตร)
61     x = np.array([0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5])
62     y = np.array([0.00, 1.05, 1.95, 3.30, 4.65, 6.40, 8.15, 10.30, 12.40, 15.05, 17.55])
63
64     degree = 2
65     print("ข้อมูลนำเข้า (การปล่อยโดรนขึ้นบิน)")
66     for xi, yi in zip(x, y):
67         print(f" t = {xi:>4} s h = {yi:>6} m")
68
69     A = build_vandermonde(x, degree)
70     AtA, Aty = normal_equation_matrices(A, y)
71     coeffs = gaussian_elimination(AtA, Aty)
72     y_pred = A @ coeffs
73     r2 = r_squared(y, y_pred)
74
75     print("\nผลลัพธ์สมการพหุนาม")
76     print(format_polynomial(coeffs))
77     # print(f"ค่าความแม่นยำของแบบจำลอง (R^2) = {r2:.6f}")
78     t_test = 6
79     h_test = predict(coeffs, t_test)
80     print(f"\nพยากรณ์ความสูงที่ t = {t_test} วินาที -> h = {round(h_test, 3)} เมตร")
81
82     plt.figure(figsize=(7, 5))
83     plt.scatter(x, y, color="crimson", label="Measured data (sensor)", zorder=3)
84     xs = np.linspace(0, 6, 200)
85     ys = predict(coeffs, xs)
86     plt.plot(xs, ys, color="navy", label="Fitted polynomial (degree 2)")
87     plt.axvline(5, color="gray", linestyle="--", linewidth=0.8)
88     plt.xlabel("Time t (seconds)")
89     plt.ylabel("Altitude h (meters)")
90     plt.title("Drone Launch: Time vs Altitude")
91     plt.legend()
92     plt.grid(alpha=0.3)
93     plt.tight_layout()
94     plt.savefig("drone.png", dpi=150)

```

อธิบายการทำงานของโปรแกรม

1. เตรียมข้อมูล (X, Y): เวลาหลังปล่อยโดรน (X) หน่วยวินาที และความสูง (Y) หน่วยเมตร จำนวน 11 จุด
2. สร้างเมทริกซ์แวนเดอร์มอนต์แปลงค่า X ให้อยู่ในรูป $[1, X, X^2]$ แล้วเรียงเป็นเมทริกซ์ A ขนาด 11×3 ด้วยฟังก์ชัน `build_vandermonde()` เพื่อใช้แทนพหุนามดีกรี 2 ที่ต้องการหา
3. คำนวณสมการปกติ : คูณเมทริกซ์ A ด้วยทรานสโพสของตัวเอง ($A^T A$) และคูณกับ Y ($A^T y$) ด้วยฟังก์ชัน `normal_equation_matrices()` เพื่อลดระบบสมการที่เกินกำหนด (11 สมการ) ให้เหลือระบบสมการ 3×3 ที่แก้ได้พอดี
4. หาค่าสัมประสิทธิ์ด้วยการกำจัดแบบเกาส์: ใช้ฟังก์ชัน `gaussian_elimination()` แก่ระบบสมการ $(A^T A)c = A^T y$ ด้วยการกำจัดตัวแปรทีละแถวแล้วแทนย้อนกลับ ได้ค่าสัมประสิทธิ์ a_0, a_1, a_2
5. สร้างฟังก์ชันทำนาย: รวมค่า a_0, a_1, a_2 เข้าเป็นฟังก์ชัน `predict()` ในรูป $Y = a_0 + a_1X + a_2X^2$
6. ทำนายผลลัพธ์: นำเวลาที่ทดสอบเข้าฟังก์ชัน `predict()` เพื่อคำนวณความสูงที่คาดว่าจะได้
7. แสดงผลและสร้างกราฟ: ประเมินความแม่นยำด้วยค่า R^2 จากฟังก์ชัน `r_squared()` แล้วพล็อตกราฟด้วย `matplotlib`

Flowchart ->



วิธีการใช้งานโปรแกรม

1. เตรียมชุดข้อมูลตามที่ผู้ใช้งานต้องการ โดยจำนวนสมาชิกใน x และ y ต้องเท่ากัน และค่าใน x ควรเรียงจากน้อยไปมาก

```
1 x = np.array([0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5])
2 y = np.array([0.00, 1.05, 1.95, 3.30, 4.65, 6.40, 8.15, 10.30, 12.40, 15.05, 17.55])
```

2. เลือกดีกรีของพหุนาม ที่ต้องการฟิต (Default degree = 2)

3. เลือกเวลาที่ต้องการทำนาย เช่น อยากรู้ว่าที่ $t = 6$ วินาที โดรนจะสูงเท่าไร ก็ปรับค่า t_test แล้วเรียกใช้ `predict()`

```
1 t_test = 6
2 h_test = predict(coeffs, t_test)
```

4. ปรับช่วงการวาดเส้นกราฟ สามารถเปลี่ยนค่าเพื่อสร้างจุดต่าง ๆ ตลอดเส้นให้ครอบคลุมกราฟ ด้วยคำสั่ง `np.linspace(ค่า X ที่น้อยสุดบนกราฟ, ค่า X ที่มากที่สุดบนกราฟ, จำนวนจุด)`

```
1 xs = np.linspace(0, 6, 200)
2 ys = predict(coeffs, xs)
```

ผลลัพธ์จากการรันโปรแกรม

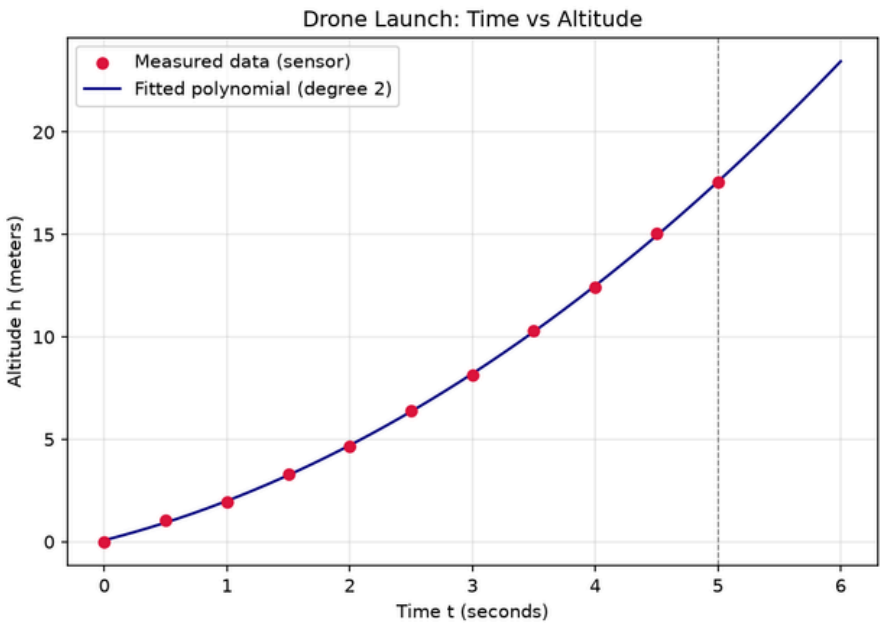
ข้อมูลนำเข้า (การปล่อยโดรนขึ้นบิน)

t = 0.0 s	h = 0.0 m
t = 0.5 s	h = 1.05 m
t = 1.0 s	h = 1.95 m
t = 1.5 s	h = 3.3 m
t = 2.0 s	h = 4.65 m
t = 2.5 s	h = 6.4 m
t = 3.0 s	h = 8.15 m
t = 3.5 s	h = 10.3 m
t = 4.0 s	h = 12.4 m
t = 4.5 s	h = 15.05 m
t = 5.0 s	h = 17.55 m

ผลลัพธ์สมการพหุนาม

y = 0.072 + 1.527x + 0.395x^2

พยากรณ์ความสูงที่ t = 6 วินาที -> h = 23.452 เมตร



ภาพนี้แสดงถึงการทำนายความสูงของโดรน ณ เวลา = 6 วินาที

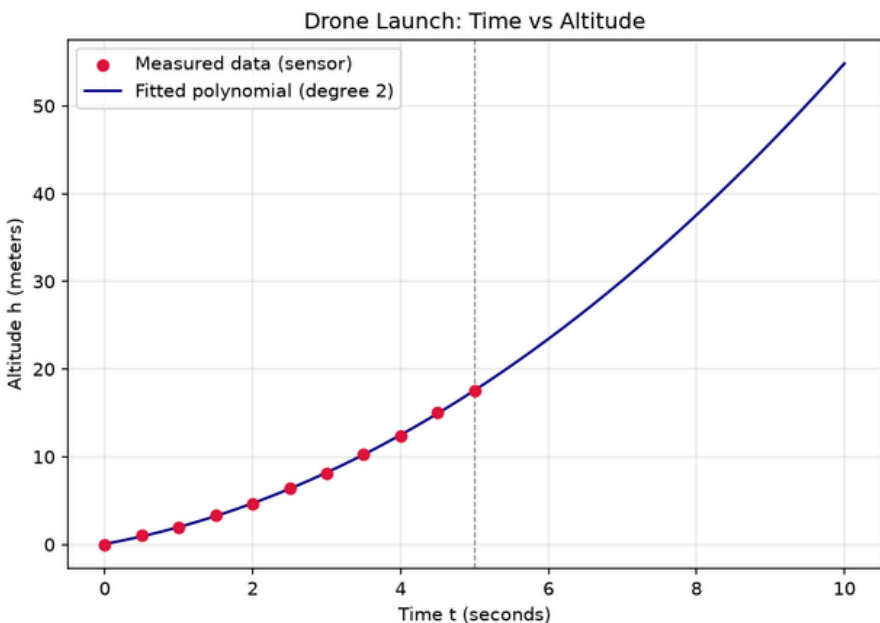
ข้อมูลนำเข้า (การปล่อยโดรนขึ้นบิน)

t = 0.0 s	h = 0.0 m
t = 0.5 s	h = 1.05 m
t = 1.0 s	h = 1.95 m
t = 1.5 s	h = 3.3 m
t = 2.0 s	h = 4.65 m
t = 2.5 s	h = 6.4 m
t = 3.0 s	h = 8.15 m
t = 3.5 s	h = 10.3 m
t = 4.0 s	h = 12.4 m
t = 4.5 s	h = 15.05 m
t = 5.0 s	h = 17.55 m

ผลลัพธ์สมการพหุนาม

y = 0.072 + 1.527x + 0.395x^2

พยากรณ์ความสูงที่ t = 10 วินาที -> h = 54.833 เมตร



ภาพนี้แสดงถึงการทำนายความสูงของโดรน ณ เวลา = 10 วินาที

ข้อสรุป

จากผลลัพธ์ของโปรแกรมจะเห็นได้ว่า หากเวลาที่ผ่านไปหลังการปล่อยโดรนขึ้นบินมากขึ้น ความสูงของโดรนก็จะเพิ่มขึ้นตามไปด้วยในอัตราที่เร่งขึ้นเรื่อย ๆ ไม่ใช่เพิ่มขึ้นแบบคงที่ซึ่งสอดคล้องกับพฤติกรรมจริงของโดรนในช่วงไต่ระดับที่ต้องเร่งมอเตอร์เพื่อเอาชนะแรงโน้มถ่วงก่อน แล้วจึงไต่ระดับได้เร็วขึ้นตามลำดับ โดยสมการพหุนามที่ได้ สามารถอธิบายความสัมพันธ์ระหว่างเวลาและความสูงได้

อย่างไรก็ตาม ค่าพยากรณ์จะมีความแม่นยำสูงเฉพาะในช่วงเวลาที่ใกล้เคียงกับข้อมูลที่เก็บจริงเท่านั้น หากนำไปพยากรณ์ในช่วงเวลาที่ไกลจากข้อมูลเดิมมากเกินไปค่าที่ได้อาจคลาดเคลื่อนสูง เนื่องจากพฤติกรรมการบินจริงของโดรนในช่วงเวลาที่ยาวนานขึ้นอาจเปลี่ยนรูปแบบไป